

Lecture 15: Recurrent Neural Networks

CS480/680 Introduction to Machine Learning
Comprehensive Study Guide with Visual Diagrams

Based on Prof. Pascal Poupart's Lecture

March 7, 2023

Contents

1	Introduction: The Problem of Variable-Length Data	3
1.1	Why Do We Need RNNs?	3
1.2	Solutions for Sequential Data	3
2	Recurrent Neural Networks: Core Concepts	3
2.1	What Makes a Network "Recurrent"?	3
2.2	Visual Architecture	4
2.3	Mathematical Formulation	4
3	Training RNNs: Backpropagation Through Time	5
3.1	How Do We Train RNNs?	5
3.2	The Challenge: Gradient Problems	5
4	RNN Architectures and Variants	6
4.1	RNN as Generalization of HMM	6
4.2	Bidirectional RNN	7
4.3	Encoder-Decoder Model (Seq2Seq)	8
5	Long Short-Term Memory (LSTM)	9
5.1	Motivation: Solving RNN Problems	9
5.2	The Power of Gates	10
5.3	LSTM Architecture	10
5.4	LSTM Equations	10
5.5	Understanding Each Component	11
6	Gated Recurrent Unit (GRU)	12
6.1	Simplifying LSTM	12
6.2	GRU Arch	13
6.3	GRU Equations	13
6.4	Understanding GRU Gates	13
6.5	LSTM vs GRU: Which to Choose?	14
7	Attention Mechanism	14
7.1	The Fundamental Problem	14
7.2	Attention Concept	15
7.3	Computing Attention	15
7.4	Decoder with Attention	15
7.5	Empirical Results	16

7.6	Attention as Word Alignment	16
8	Recursive Neural Networks	16
8.1	From Sequences to Trees	16
8.2	Visual Comparison	17
8.3	Applications in NLP	17
8.4	Sentiment Analysis Example	17

1 Introduction: The Problem of Variable-Length Data

1.1 Why Do We Need RNNs?

Traditional feed-forward neural networks have a **fixed architecture** that requires **fixed-length input vectors**. This creates a fundamental problem for many real-world applications:

- **Time-series data:** Stock prices, sensor measurements (varying duration)
- **Natural language:** Sentences with different word counts
- **Speech signals:** Variable length audio
- **Video sequences:** Different frame counts

The Fixed-Size Problem

If input size changes, a fixed architecture cannot accommodate it. For example:

- Sentence 1: “I love ML” (3 words) → needs 3 input nodes
- Sentence 2: “Deep learning is fascinating” (4 words) → needs 4 input nodes

A traditional feedforward network cannot handle both!

1.2 Solutions for Sequential Data

1. **Convolutional Neural Networks:** For spatial data (images)
2. **Recurrent Neural Networks:** For sequential/temporal data
3. **Graph Neural Networks:** For arbitrary graph structures
 - Recursive NNs are special cases for tree structures

Hierarchy of Models

Hidden Markov Models (HMM) $\subset$ Recurrent Neural Networks (RNN) $\subset$ Recursive Neural Networks $\subset$ Graph Neural Networks

Each generalization handles more complex structures while maintaining the ability to process variable-length data.

2 Recurrent Neural Networks: Core Concepts

2.1 What Makes a Network “Recurrent”?

Recurrent Neural Network

An RNN is a neural network where **outputs of certain nodes are fed back as inputs** to the same (or previous) nodes, creating cycles in the computational graph.

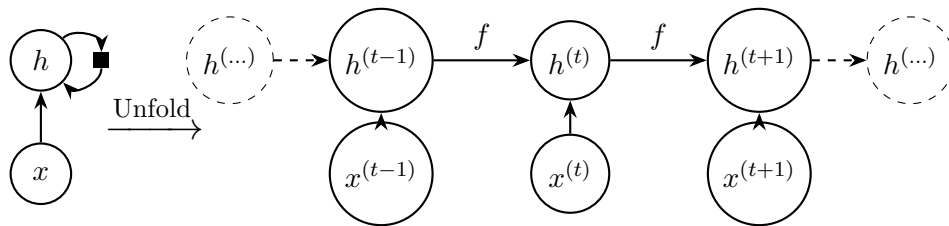
The Key Idea

At each time step t :

1. Receive input x_t
2. Use previous hidden state h_{t-1} as “memory”
3. Compute new hidden state $h_t = f(h_{t-1}, x_t)$
4. The hidden state encodes information from ALL previous inputs

Think of it as: “The network has memory of what it has seen before.”

2.2 Visual Architecture



2.3 Mathematical Formulation

The recurrent relationship is:

$$\begin{aligned} h_t &= f(h_{t-1}, x_t) && \text{(hidden state update)} \\ y_t &= g(h_t) && \text{(output computation)} \end{aligned}$$

A concrete example with sigmoid activation:

$$h_t = \sigma(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

where:

- W_{hh} : Recurrent weight matrix (hidden-to-hidden)
- W_{xh} : Input weight matrix
- W_{hy} : Output weight matrix
- b_h, b_y : Bias vectors
- σ : Activation function (sigmoid, tanh, ReLU)

Weight Sharing: The Magic of RNNs

The **same function** f with the **same weights** is applied at every time step. This is crucial for:

1. **Handling variable-length sequences:** Works for any sequence length
2. **Learning temporal patterns:** Same pattern recognition across time
3. **Parameter efficiency:** Number of parameters independent of sequence length

Without weight sharing, you'd need different weights for each time step $\rightarrow$ can't generalize to new sequence lengths!

3 Training RNNs: Backpropagation Through Time

3.1 How Do We Train RNNs?

Backpropagation Through Time (BPTT)

Training method where:

1. **Unroll** the RNN for a fixed number of time steps
2. Treat the unrolled network as a **feedforward network**
3. Apply standard **backpropagation**
4. **Combine gradients** from shared weights

Why BPTT Works

The recurrent structure becomes a deep feedforward network when unrolled. Since the same function f appears multiple times, we compute the gradient at each occurrence and sum them up for the final update.

3.2 The Challenge: Gradient Problems

Gradient Vanishing

When backpropagating through time:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_T} \prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}}$$

If each factor $\frac{\partial h_t}{\partial h_{t-1}} < 1$, the product shrinks exponentially:

$$0.9^{50} \approx 0.005 \quad (\text{vanishes!})$$

Consequence: Network cannot learn long-range dependencies because gradients from early time steps become too small to update weights effectively.

Gradient Explosion

Conversely, if factors are > 1 :

$$1.1^{50} \approx 117.4 \quad (\text{explodes!})$$

Why RNNs are special: The **same** weights are used at every step, so the same eigenvalues get multiplied repeatedly. In regular feedforward networks, different layers have different weights, so this doesn't happen as severely.

Long-Range Dependencies

Information from early inputs (x_1, x_2) gradually gets **forgotten** as the hidden state is modified at each subsequent time step. The network struggles to remember information over long sequences (e.g., 50+ time steps).

Prediction Drift

When predicting multiple steps ahead (e.g., weather forecasting):

- Errors at step t accumulate
- Affect predictions at steps $t + 1, t + 2, \dots$
- Predictions drift further from true values over time

4 RNN Architectures and Variants

4.1 RNN as Generalization of HMM

HMM vs RNN

Hidden Markov Model (HMM):

- Edges represent **probabilistic dependencies**
- Discrete hidden states
- Limited expressiveness

Recurrent Neural Network (RNN):

- Edges represent **functional dependencies**
- Continuous hidden states (vectors)
- More expressive (universal function approximator)

Causal Structure

In activity recognition:

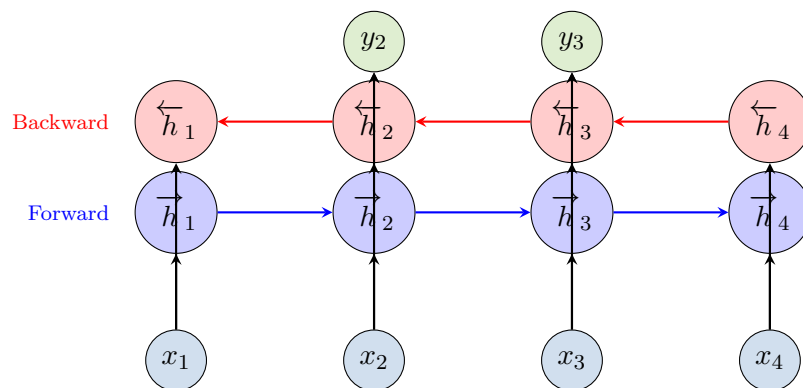
- **True causality:** Activity $y \rightarrow$ Sensor measurement x
- **Inference direction:** Observe $x \rightarrow$ Infer y

This is why in diagrams, arrows point from y to x (showing causality), even though we compute in the opposite direction (inference).

4.2 Bidirectional RNN

Bidirectional RNN

Architecture that processes sequences in **both forward and backward directions**, allowing each output to depend on both past and future context.



Key insight for y_2 :

- $\vec{h}_2$ captures info from x_1, x_2 (past)
- $\overleftarrow{h}_2$ captures info from x_3, x_4 (future)
- $y_2 = g(\vec{h}_2, \overleftarrow{h}_2)$ depends on entire sequence!

When to Use Bidirectional RNN

Use when the **entire sequence is available before prediction**:

- **Speech recognition:** Process entire utterance, then transcribe
- **Machine translation:** Read complete sentence first
- **Named entity recognition:** Context before and after helps

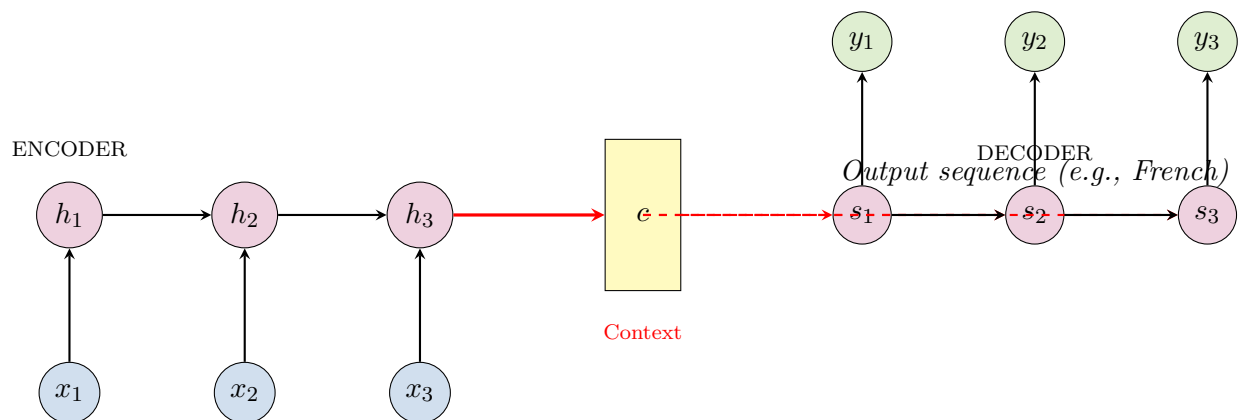
Don't use for real-time applications where you need predictions as data arrives (e.g., live speech synthesis).

4.3 Encoder-Decoder Model (Seq2Seq)

Sequence-to-Sequence Model

Architecture with two RNNs:

1. **Encoder:** Compresses input sequence into fixed-size context vector c
2. **Decoder:** Generates output sequence from context vector



Input sequence (e.g., English)

Applications:

- **Machine translation:** Different input/output lengths
- **Question answering:** Question $\rightarrow$ Answer
- **Dialogue systems:** Message $\rightarrow$ Response
- **Text summarization:** Long text $\rightarrow$ Summary

Human Translation Analogy

Process is like human translation:

1. **Read** the sentence in the source language
2. **Put the page away** and remember the meaning in your head (context vector c)
3. **Write** the translation word by word, keeping the original meaning in mind

The context vector c (typically 500-1000 dimensions) is your “memory” of what you read.

Why Feed c at Every Decoder Step?

The context vector is fed into the decoder at **every time step** because:

- Prevents forgetting what we're translating
- Like periodically checking your memory
- Without it, the meaning might get lost as s_t evolves
- Mitigates long-range dependency problem

Decoder equations:

$$s_t = f(s_{t-1}, y_{t-1}, c) \quad (1)$$

$$y_t = g(s_t, c) \quad (2)$$

Historical Impact

In 2014, machine translation was **revolutionized**:

- **Before:** Multi-year research pipelines with teams
- **After:** Single researcher + desktop + 1 week

This was a watershed moment in NLP history!

5 Long Short-Term Memory (LSTM)

5.1 Motivation: Solving RNN Problems

LSTM addresses three critical challenges:

1. Gradient vanishing
2. Gradient explosion
3. Long-range dependencies

5.2 The Power of Gates

Gates in LSTM

A **gate** is a neural network layer with sigmoid activation (σ):

$$\text{Gate output} = \sigma(\dots) \in [0, 1]$$

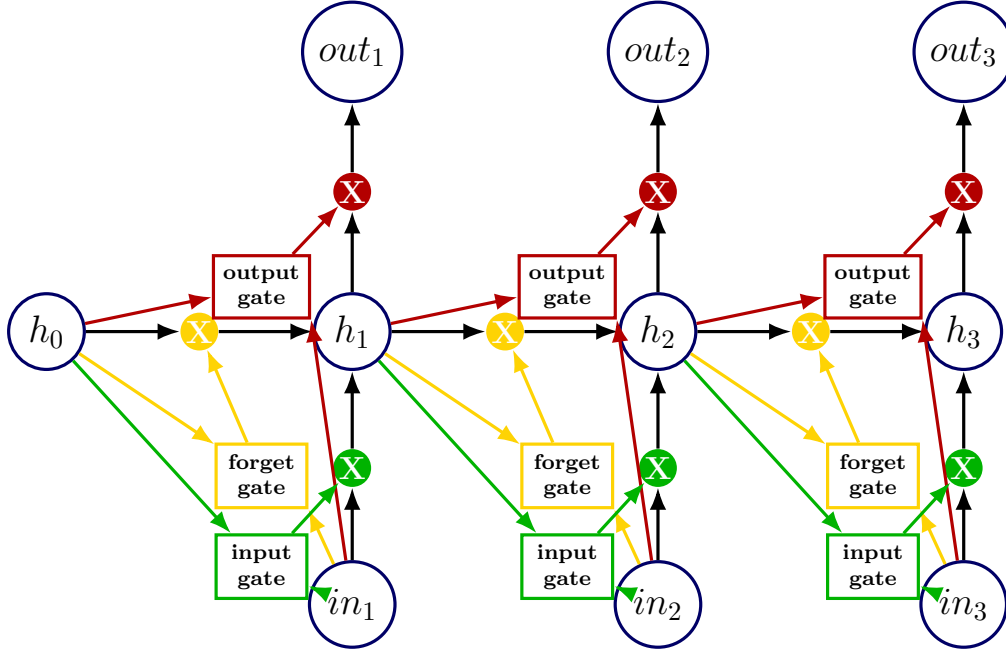
Function:

- Gate = 0: Block information completely (close gate)
- Gate = 1: Allow information to pass freely (open gate)
- $0 < \text{Gate} < 1$: Partially allow information

Mechanism: Multiply the gate value with the information:

$$\text{Filtered info} = \text{Gate} \odot \text{Information}$$

5.3 LSTM Architecture



5.4 LSTM Equations

The LSTM unit computes:

$$\text{Forget gate: } f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f) \quad (3)$$

$$\text{Input gate: } i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i) \quad (4)$$

$$\text{Candidate cell: } \tilde{c}_t = \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c) \quad (5)$$

$$\text{Cell state: } c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (6)$$

$$\text{Output gate: } o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o) \quad (7)$$

$$\text{Hidden state: } h_t = o_t \odot \tanh(c_t) \quad (8)$$

where $\odot$ denotes element-wise (Hadamard) multiplication.

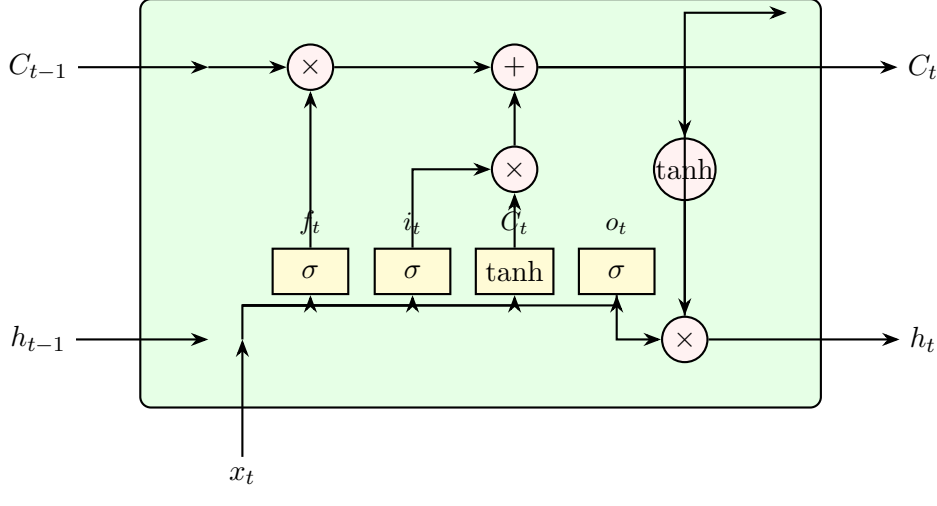


Figure 1: LSTM Cell Internal Architecture[cite: 195]. f_t : forget gate, i_t : input gate, o_t : output gate.

5.5 Understanding Each Component

- **Forget gate (f_t):** Decides what to *discard* from previous cell state c_{t-1}
 - $f_t \approx 1$: Keep previous memory
 - $f_t \approx 0$: Forget previous memory
- **Input gate (i_t):** Decides what new info to *store* in cell state
 - $i_t \approx 1$: Accept new input
 - $i_t \approx 0$: Ignore new input
- **Candidate cell ($\tilde{C}_t$):** *Proposes* new values for cell state (using $\tanh$ to bound in $[-1, 1]$)
- **Cell state update:** Combines forget and input:

$$c_t = \underbrace{f_t \odot c_{t-1}}_{\text{what to keep}} + \underbrace{i_t \odot \tilde{C}_t}_{\text{what to add}}$$

- **Output gate (o_t):** Decides what to *output* from cell state
- **Hidden state:** Filtered version of cell state (what the network “sees”)

Why LSTM Solves Gradient Problems

Gradient flow through cell state:

- Cell state c_t acts as a “memory highway”
- Addition operation (not multiplication) in cell update preserves gradients
- Gates regulate signal magnitude, preventing explosion
- $\tanh$ bounds values to $[-1, 1]$, preventing unbounded growth

For long-range memory:

- To preserve memory: $f_t = 1$ (don’t forget), $i_t = 0$ (don’t add new)
- To update memory: $f_t = 0$ (forget old), $i_t = 1$ (add new)
- Network learns optimal gating strategy during training

Human Memory Analogy

Think of LSTM like your working memory:

- **Forget gate:** “Should I forget this old information?”
- **Input gate:** “Should I pay attention to this new information?”
- **Output gate:** “What should I tell others from what I know?”

The network learns when to remember, when to forget, and what to output!

6 Gated Recurrent Unit (GRU)

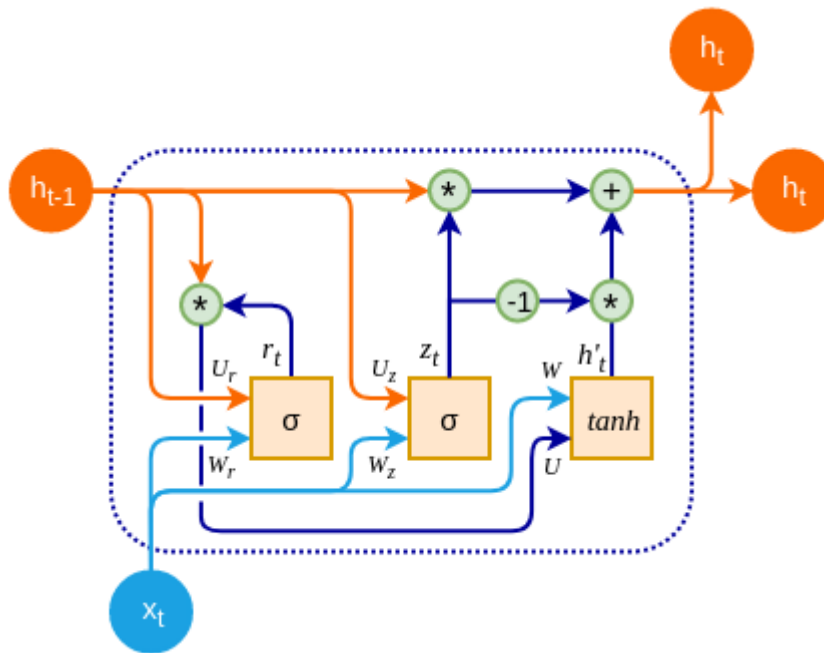
6.1 Simplifying LSTM

GRU

A **simplified version** of LSTM with:

- Only **2 gates** (instead of 3)
- **No separate cell state** (only hidden state)
- **Fewer parameters**
- Often similar performance to LSTM

6.2 GRU Arch



6.3 GRU Equations

$$\text{Reset gate: } r_t = \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \quad (9)$$

$$\text{Update gate: } z_t = \sigma(W_{xz}x_t + W_{hz}h_{t-1} + b_z) \quad (10)$$

$$\text{Candidate: } \tilde{h}_t = \tanh(W_{xh}x_t + r_t \odot (W_{hh}h_{t-1}) + b_h) \quad (11)$$

$$\text{Hidden state: } h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (12)$$

6.4 Understanding GRU Gates

- **Update gate (z_t):** Controls the *tradeoff* between old and new
 - $z_t = 0$: Keep old state h_{t-1} entirely
 - $z_t = 1$: Use new candidate $\tilde{h}_t$ entirely
 - Intermediate: Convex combination (weighted average)
- **Reset gate (r_t):** Controls how much *previous hidden state* influences the candidate
 - $r_t = 0$: Ignore previous state (“reset” memory)
 - $r_t = 1$: Use full previous state

GRU Design Philosophy

Instead of having separate mechanisms for forgetting and adding (like LSTM), GRU uses a **single update gate** that makes a decision:

“How much of the new candidate should I use vs. how much of the old state should I keep?”

This is simpler but often just as effective!

6.5 LSTM vs GRU: Which to Choose?

Aspect	LSTM	GRU
Gates	3 (forget, input, output)	2 (reset, update)
Cell state	Separate c_t	Only h_t
Parameters	More	Fewer
Training time	Slower	Faster
Expressiveness	More expressive	Simpler
Performance	Problem-dependent	Problem-dependent

Practical Advice

- **Try both:** Performance is problem-dependent
- **Start with GRU:** Faster to train, often sufficient
- **Use LSTM if:** You have lots of data and need maximum expressiveness
- **Modern trend:** Transformers have largely replaced both for NLP

7 Attention Mechanism

7.1 The Fundamental Problem

Fixed Context Vector Limitation

Even with LSTM/GRU, using a **single fixed context vector** c to summarize a long sequence has problems:

- Information bottleneck (compress entire input into one vector)
- Memory gets perturbed at each step
- Early information gets “washed out”
- Especially bad for long sequences (30+ words)

Human Translation

When you translate a document, you **don't**:

- × Memorize the entire document
- × Put it away completely
- × Write the translation from memory

Instead, you:

- ✓ Read and translate incrementally
- ✓ Look back at specific relevant parts
- ✓ Focus on different parts for different output words

Attention mimics this behavior!

7.2 Attention Concept

Attention Mechanism

Instead of a single context vector, compute a **different context vector** c_t at each decoder step by taking a **weighted combination** of all encoder hidden states:

$$c_t = \sum_{j=1}^T \alpha_{tj} h_j$$

The weights α_{tj} indicate which input positions are most relevant for the current output.

7.3 Computing Attention

Three-step process at decoder time t :

1. **Compute alignment scores:** How well does decoder state s_{t-1} align with each encoder state h_j ?

$$e_{tj} = \text{align}(s_{t-1}, h_j) = s_{t-1}^T h_j \quad (\text{dot product})$$

Other alignment functions exist (additive, multiplicative, etc.)

2. **Compute attention weights:** Normalize scores to probabilities via softmax

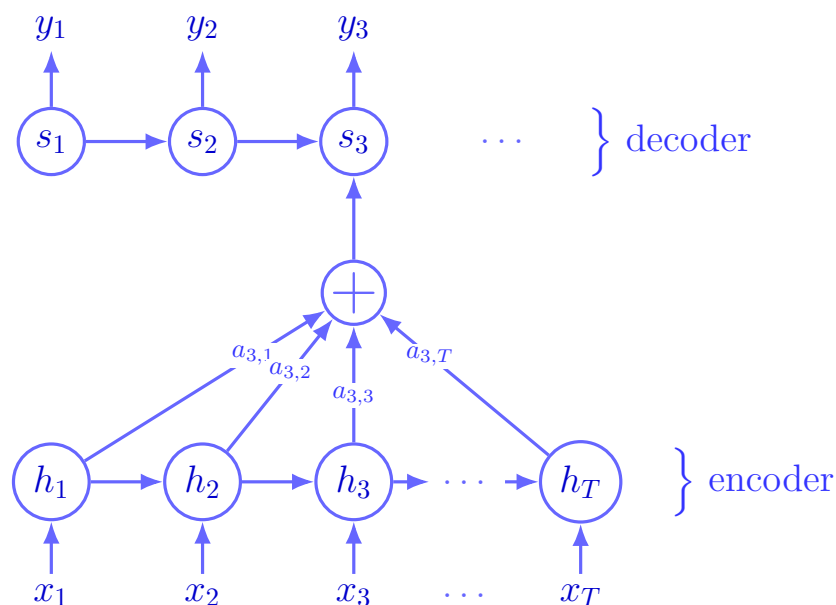
$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^T \exp(e_{tk})}$$

Now $\sum_{j=1}^T \alpha_{tj} = 1$ (convex combination)

3. **Compute context vector:** Weighted sum of encoder hidden states

$$c_t = \sum_{j=1}^T \alpha_{tj} h_j$$

7.4 Decoder with Attention



Why Attention Works

1. **Selective focus:** Each output word can “attend to” relevant input words
2. **No recency bias:** All input words are equally accessible (no gradual forgetting)
3. **Handles word reordering:** Different languages have different word orders—attention learns the alignment
4. **Interpretability:** Attention weights α_{tj} show which input words influenced each output word
5. **Long sequences:** Performance doesn’t degrade with length

7.5 Empirical Results

Machine Translation Performance (BLEU score):

Key observation: Attention shows the **biggest improvement** for long sequences (30+ words).

7.6 Attention as Word Alignment

Attention weights often correspond to linguistic alignment between source and target words, even though alignment is **learned implicitly** (not supervised)!

Example from Bahdanau et al. (2015): Translating French to English, attention weights show which French words align with each English word—matching human intuition about word correspondence.

8 Recursive Neural Networks

8.1 From Sequences to Trees

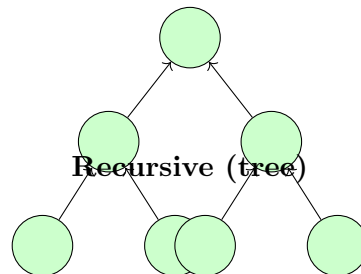
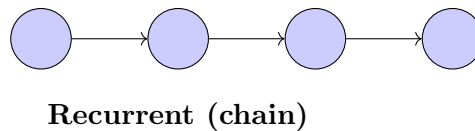
Recursive Neural Network

Neural networks that operate on **tree or graph structures**, not just sequences. Weight sharing allows handling trees of different sizes/shapes.

Relationship hierarchy:

- **Recurrent NN:** Handle chains/sequences
- **Recursive NN:** Handle trees (special case of graphs)
- **Graph NN:** Handle arbitrary graphs

8.2 Visual Comparison



8.3 Applications in NLP

Key question: What tree structure should we use?

Common choice: Linguistic parse trees

- **Constituency parsing:** Hierarchical phrase structure
- **Dependency parsing:** Grammatical dependencies

Applications:

- **Sentiment analysis:** Compose word sentiments hierarchically
- **Semantic parsing:** Understand compositional meaning
- **Sentence embeddings:** Create fixed-size vector from variable-length sentence

8.4 Sentiment Analysis Example

Recursive NNs compose word embeddings according to sentence structure:

1. Each word has an embedding vector
2. Combine child embeddings $\rightarrow$ parent node embedding
3. Root embedding represents entire sentence